

MODULE 1-2

Introduction to Software
Packages and Regular
Expressions , Text processing

Introduction to Software Packages

1. spaCy

- spaCy is an open-source software library for advanced natural language processing in Python.
- It is designed specifically for production use and helps build applications that process large volumes of text.
- Key features include tokenization, part-of-speech tagging, dependency parsing, named entity recognition, and word vectors.

python

```
import spacy  
nlp = spacy.load("en_core_web_sm")  
doc = nlp("I love Natural Language Processing.")  
for token in doc:  
    print(token.text, token.pos_)
```

Introduction to Software Packages

2.NLTK (Natural Language Toolkit)

- NLTK is a leading platform for building Python programs to work with human language data.
- It provides easy-to-use interfaces to over 50 corpora and lexical resources, along with text-processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning.

python

```
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt')
text = "NLTK is a leading platform for building Python programs."
print(word_tokenize(text))
```

Introduction to Software Packages

3. Gensim

- Gensim is a Python library for topic modeling and document similarity analysis.
- It uses modern statistical machine learning to extract semantic topics from documents.
- It is particularly known for its efficiency in handling large text collections using algorithms like Word2Vec, Doc2Vec, and Latent Dirichlet Allocation (LDA).

python

```
from gensim.models import Word2Vec  
sentences = [["machine", "learning"], ["deep", "learning"]]  
model = Word2Vec(sentences, vector_size=50, min_count=1, workers=4)  
print(model.wv["machine"])
```

Introduction to Software Packages

4. PyTorch

- PyTorch is an open-source machine learning framework based on Torch.
- It provides tools for deep learning and tensor computation with GPU acceleration.
- It is widely used in NLP, computer vision, and generative models due to its flexibility, dynamic computation graph, and strong community support.

python

```
import torch
x = torch.tensor([2.0], requires_grad=True)
y = x ** 2 + 3 * x + 5
y.backward()
print(x.grad)  # derivative of y wrt x
```

2. Regular Expressions

Regular expressions (regex) are sequences of characters that define search patterns. They are extremely useful for pattern matching, text searching, and string manipulation.

2.1 Importance of Regular Expressions

- Simplify complex string searching and manipulation.
- Used in data validation (e.g., email, phone numbers).
- Help in text preprocessing for NLP tasks.
- Widely used in compilers, interpreters, and search engines.

2.2 Properties of Regular Expressions

- Powerful for defining string patterns.
- Provide flexibility for matching repetitions, optional elements, and character classes.
- Support grouping and alternation.
- Allow efficient searching in large text corpora.

2.3 Working of Regular Expressions

Regular expressions use metacharacters like ., *, +, ?, ^, \$, and [] to build patterns. For example:

- '^a' matches strings starting with 'a'.
- 'a\$' matches strings ending with 'a'.
- '[0-9]+' matches one or more digits.
- 'colou?r' matches 'color' or 'colour'.

2.4 Python Package (re)

Python provides the 're' module for working with regular expressions. Key functions include:

- `re.match()`: checks for a match only at the beginning of the string.
- `re.search()`: searches for a match anywhere in the string.
- `re.findall()`: returns all non-overlapping matches.
- `re.sub()`: replaces occurrences of a pattern with a string.

3. Case Study: Working of Eliza Chatbot

The Eliza chatbot, developed by Joseph Weizenbaum in the 1960s, is one of the earliest NLP systems. It simulates a psychotherapist by recognizing keywords and pattern-matching user input using regular expressions.

Key Characteristics:

- Identifies patterns in user text using regex rules.
- Uses substitution rules to generate responses.
- For example, if a user says 'I feel sad', Eliza matches the pattern 'I feel (.*)' and responds with 'Why do you feel \1?'
- Demonstrates the power of regex in building conversational agents.

Python RegEx

- A RegEx, or Regular Expression, is a sequence of characters that forms a search pattern.
- RegEx can be used to check if a string contains the specified search pattern.

RegEx Module

- Python has a built-in package called re, which can be used to work with Regular Expressions.
- Import the re module:

```
import re
```

RegEx in Python

- When you have imported the re module, you can start using regular expressions:

Example

- Search the string to see if it starts with "The" and ends with "Spain":

```
import re
```

```
txt = "The rain in Spain"
```

```
x = re.search("^The.*Spain$", txt)
```


RegEx Functions

- The re module offers a set of functions that allows us to search a string for a match:

Function	Description
findall	Returns a list containing all matches
search	Returns a Match object if there is a match anywhere in the string
split	Returns a list where the string has been split at each match
sub	Replaces one or many matches with a string

Metacharacters

- Metacharacters are characters with a special meaning:

Character	Description	Example
[]	A set of characters	"[a-m]"
\	Signals a special sequence (can also be used to escape special characters)	"\d"
.	Any character (except newline character)	"he..o"
^	Starts with	"^hello"
\$	Ends with	"planet\$"
*	Zero or more occurrences	"he.*o"
+	One or more occurrences	"he.+o"
?	Zero or one occurrences	"he.?o"
{}	Exactly the specified number of occurrences	"he{2}o"
	Either or	"falls stays"
()	Capture and group	

Special Sequences

- A special sequence is a `\` followed by one of the characters in the list below, and has a special meaning:

Character	Description	Example
<code>\A</code>	Returns a match if the specified characters are at the beginning of the string	<code>"\AThe"</code>
<code>\b</code>	Returns a match where the specified characters are at the beginning or at the end of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")	<code>r"\bain"</code> <code>r"ain\b"</code>
<code>\B</code>	Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")	<code>r"\Bain"</code> <code>r"ain\B"</code>
<code>\d</code>	Returns a match where the string contains digits (numbers from 0-9)	<code>"\d"</code>
<code>\D</code>	Returns a match where the string DOES NOT contain digits	<code>"\D"</code>
<code>\s</code>	Returns a match where the string contains a white space character	<code>"\s"</code>
<code>\S</code>	Returns a match where the string DOES NOT contain a white space character	<code>"\S"</code>
<code>\w</code>	Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore <code>_</code> character)	<code>"\w"</code>
<code>\W</code>	Returns a match where the string DOES NOT contain any word characters	<code>"\W"</code>
<code>\Z</code>	Returns a match if the specified characters are at the end of the string	<code>"Spain\Z"</code>

The findall() Function

- The findall() function returns a list containing all matches.

Example

- Print a list of all matches:

```
import re  
  
txt = "The rain in Spain"  
x = re.findall("ai", txt)  
  
print(x)          #['ai', 'ai']
```

- The list contains the matches in the order they are found.
- If no matches are found, an empty list is returned:

- Example
- Return an empty list if no match was found:

```
import re
```

```
txt = "The rain in Spain"  
x = re.findall("Portugal", txt)  
print(x)          #[ ]
```

The search() Function

- The search() function searches the string for a match, and returns a Match object if there is a match.
- If there is more than one match, only the first occurrence of the match will be returned:

Example

- Search for the first white-space character in the string:

```
import re
```

```
txt = "The rain in Spain"
```

```
x = re.search("\s", txt)
```

```
print("The first white-space character is located in position:", x.start())
```

- If no matches are found, the value None is returned:

Example

- Make a search that returns no match:

```
import re
```

```
txt = "The rain in Spain"
```

```
x = re.search("Portugal", txt)
```

```
print(x)
```


The split() Function

- The split() function returns a list where the string has been split at each match:

Example

- Split at each white-space character:

```
import re

txt = "The rain in Spain"
x = re.split("\s", txt)
print(x)
```

- You can control the number of occurrences by specifying the maxsplit parameter:

Example

- Split the string only at the first occurrence:

```
import re
```

```
txt = "The rain in Spain"
```

```
x = re.split("\s", txt, 1)
```

```
print(x)
```

The sub() Function

- The sub() function replaces the matches with the text of your choice:

Example

- Replace every white-space character with the number 9:

```
import re
```

```
txt = "The rain in Spain"  
x = re.sub("\s", "9", txt)  
print(x)
```

- You can control the number of replacements by specifying the count parameter:

Example

- Replace the first 2 occurrences:

```
import re
```

```
txt = "The rain in Spain"
```

```
x = re.sub("\s", "9", txt, 2)
```

```
print(x)
```

Match Object

- A Match Object is an object containing information about the search and the result.
- Note: If there is no match, the value None will be returned, instead of the Match Object.

Example

- Do a search that will return a Match Object:

```
import re
```

```
txt = "The rain in Spain"
```

```
x = re.search("ai", txt)
```

```
print(x) #this will print an object
```

Output

```
<re.Match object; span=(5, 7), match='ai'>
```

- The Match object has properties and methods used to retrieve information about the search, and the result:

- `.span()` -returns a tuple containing the start-, and end positions of the match.

- `.string`- returns the string passed into the function

- `.group()`-returns the part of the string where there was a match

Example

- Print the position (start- and end-position) of the first match occurrence.
- The regular expression looks for any words that starts with an upper case "S":

```
import re
```

```
txt = "The rain in Spain"
```

```
x = re.search(r"\bS\w+", txt)
```

```
print(x.span())    # (12, 17)
```


Example

- Print the string passed into the function:

```
import re
```

```
txt = "The rain in Spain"
```

```
x = re.search(r"\bS\w+", txt)
```

```
print(x.string)
```

Output

The rain in Spain

Example

- Print the part of the string where there was a match.
- The regular expression looks for any words that starts with an upper case "S":

```
import re
```

```
txt = "The rain in Spain"  
x = re.search(r"\bS\w+", txt)  
print(x.group()) #Spain
```

Named Groups with Regular Expressions

- Groups are used in Python in order to reference regular expression matches.
- By default, groups, without names, are referenced according to numerical order starting with 1 .
- Let's say we have a regular expression that has 3 subexpressions.
- A user enters in his birthdate, according to the month, day, and year.
- Let's say the user must first enter the month, then the day, and then the year.

Named Groups with Regular Expressions

- Using the `group()` function in Python, without named groups, the first match (the month) would be referenced using the statement, `group(1)`.
- The second match (the day) would be referenced using the statement, `group(2)`.
- The third match (the year) would be referenced using the statement, `group(3)`.

Named Groups with Regular Expressions

- Now, with named groups, we can name each match in the regular expression.
- So instead of referencing matches of the regular expression with numbers (group(1), group(2), etc.), we can reference matches with names, such as group('month'), group('day'), group('year').
- Named groups makes the code more organized and more readable.

Named Groups with Regular Expressions

- By seeing, `group(1)`, you don't really know what this represents.
- But if you see, `group('month')` or `group('year')`, you know it's referencing the month or the year.
- So named groups makes code more readable and more understandable rather than the default numerical referencing.

Example

```
>>> import re
>>> string1= "June 15, 1987"
>>> regex= r"^(?P<month>\w+)\s(?P<day>\d+)\s(?P<year>\d+)"
>>> matches= re.search(regex, string1)
>>> print("Month: ", matches.group('month'))
>>> print("Day: ", matches.group('day'))
>>> print("Year: ", matches.group('year'))
```

Output

Month: June

Day: 15

Year: 1987

Understanding Pattern Objects

Match Method Vs Search Method

- In Python's 're' module, both `match()` and `search()` are used to check for patterns in a string.
 - `match()`: Checks for a match only at the beginning of the string.
 - `search()`: Searches the entire string and returns the first match found.

Example:

```
re.match('a', 'abc') # Matches because 'a' is at the start
```

```
re.search('b', 'abc') # Matches because 'b' is found in the string
```

Understanding Pattern Objects

- Finditer Method

The `finditer()` method returns an iterator yielding match objects for all non-overlapping matches. It allows iteration over all matches with detailed information.

Example:

```
for match in re.finditer(r'\d+', '12 drummers, 11  
pipers, 10 lords'):  
    print(match.group()) # Prints 12, 11, 10
```

Understanding Pattern Objects

- Logical OR

The logical OR operator is represented by the pipe symbol (|) in regular expressions.

It allows matching one pattern OR another.

Example:

```
re.search('cat|dog', 'I have a dog') # Matches  
'dog'
```

```
re.search('cat|dog', 'I have a cat') # Matches 'cat'
```

Understanding Pattern Objects

● Beginning and End Patterns

To specify positions in strings:

- ^ matches the beginning of a string.
- \$ matches the end of a string.

Example:

`re.search('^Hello', 'Hello World')` # Matches
because 'Hello' is at the start

`re.search('World$', 'Hello World')` # Matches
because 'World' is at the end

Understanding Pattern Objects

Parenthesis

Parentheses () are used for grouping and capturing in regular expressions. They help in extracting specific sub-patterns from matches.

Example:

```
match = re.search('(\d+)-(\d+)', '2025-09')
print(match.group(0)) # 2025-09
print(match.group(1)) # 2025
print(match.group(2)) # 09
```

String Modification

- **String Modification in Python (Using re Module)**
- In Python, string modification using the 're' (regular expressions) module provides powerful tools to manipulate and transform text. Three commonly used methods are: `split()`, `sub()`, and `subn()`. These methods allow splitting strings, substituting substrings, and performing substitutions while also returning the number of replacements.

1. split() Method

The `re.split()` method is used to split a string into a list using a pattern as the delimiter.

It is similar to the built-in `str.split()`, but more flexible because it accepts regex patterns.

Syntax:

```
re.split(pattern, string, maxsplit=0)
```

Parameters:

- pattern: The regex pattern used as a delimiter.
- string: The input string to be split.
- maxsplit: (Optional) Maximum number of splits.

Example:

```
import re
text = 'one,two;three four'
result = re.split(r'[; ,]', text)
print(result) # ['one', 'two', 'three', 'four']
```

2. sub() Method

The `re.sub()` method is used to replace occurrences of a pattern with a specified replacement string.

It is widely used for text cleaning and transformation.

Syntax:

```
re.sub(pattern, repl, string, count=0)
```

Parameters:

- pattern: The regex pattern to find.
- repl: Replacement string or function.
- string: The input string.
- count: (Optional) Maximum number of replacements (default = all).

Example:

```
import re
text = 'blue sky and blue ocean'
result = re.sub(r'blue', 'green', text)
print(result) # 'green sky and green ocean'
```

3. subn() Method

The re.subn() method works similarly to sub(), but returns a tuple containing:

1. The modified string after replacements.
2. The number of substitutions made.

Syntax:

```
re.subn(pattern, repl, string, count=0)
```

Example:

```
import re
text = 'apple orange apple banana'
result = re.subn(r'apple', 'mango', text)
print(result) # ('mango orange mango banana', 2)
```

Here, two replacements were performed, so the count returned is 2.

The `split()`, `sub()`, and `subn()` methods in the 're' module are essential for string modification tasks.

- `split()` helps in dividing strings using complex delimiters.
- `sub()` performs replacements within strings.
- `subn()` provides both the modified string and the number of replacements.

Text Processing and Sentiment Classification

Text processing is a fundamental step in Natural Language Processing (NLP). It involves converting raw text into structured formats for computational models. This lecture note covers important concepts such as tokens, corpus, vocabulary, and techniques like tokenization, encoding, feature computation, and evaluation. We also explore advanced concepts such as sentiment classification, tokenization strategies, and string matching methods like minimum edit distance.

1. Words, Tokens, Vocabulary, and Corpus

Word : A meaningful linguistic unit (e.g., 'apple').

Token : The occurrence of a word in a text. For example, 'apple apple' has two tokens.

Vocabulary : The set of unique tokens extracted from a text or corpus.

Corpus : A large collection of texts used for NLP tasks.

Example:

Text: 'I love NLP. NLP is amazing.'

Tokens: ['I', 'love', 'NLP', 'NLP', 'is', 'amazing']

Vocabulary: {'I', 'love', 'NLP', 'is', 'amazing'}

2. Tokenization in spaCy

Tokenization is the process of splitting text into words, phrases, or symbols.

spaCy provides efficient tokenization for multiple languages.

Example:

```
import spacy
nlp = spacy.load('en_core_web_sm')
doc = nlp('I love NLP!')
for token in doc:
    print(token.text)
```

Output: I, love, NLP, !

3. Sentiment Classification (Yelp Reviews)

Sentiment classification involves predicting the sentiment (positive/negative) of reviews.

Steps:

1. Download Dataset : Yelp reviews dataset.
2. Data Preparation : Using NumPy, pandas, re, and Counter.
3. Build Vocabulary: From tokens, Data-Frames, or corpus.
4. Encoding: One-hot encoding or other methods.
5. Train-Test Split: Divide data for training and evaluation.
6. Feature Computation: Convert text into numerical vectors.
7. Confusion Matrix: Evaluate predictions.
8. Analysis: Identify accuracy, precision, recall, F1-score.

4. Encoding Techniques

- One-Hot Encoding: Represent each word as a binary vector.
 - Encoding Documents: Convert each document into a sequence of vectors.
 - Example : Vocabulary = {good, bad, amazing}
- Text: 'good bad' $\rightarrow$ [1, 1, 0]

5. Train-Test Split and Feature Computation

Splitting data ensures unbiased evaluation.

Example: `sklearn.model_selection.train_test_split()`

Feature computation involves transforming raw text into TF-IDF, Bag-of-Words, or embeddings.

6. Confusion Matrix and Analysis

A confusion matrix evaluates classification models:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Metrics:

- Accuracy = $(TP+TN)/(TP+FP+TN+FN)$
- Precision = $TP/(TP+FP)$
- Recall = $TP/(TP+FN)$
- F1-score = $2 * (Precision * Recall) / (Precision + Recall)$

7. Language Independent Tokenization

- **Word Tokenization**: Splits by words.
 - Problem: Out-of-vocabulary words, multiple forms.
- **Character Tokenization**: Splits by characters.
 - Drawback: Sequence becomes too long.
- **Sub-Word Tokenization**: Splits words into meaningful subunits.
 - Example: 'playing' → 'play' + 'ing'.
 - Problem: Needs balance between vocabulary size and sequence length.
- **Byte Pair Encoding (BPE)**: A compression-based algorithm for sub-word tokenization.

8. String Matching and Spelling Correction

String matching techniques are used in spelling correction and approximate string search.

- **Minimum Edit Distance**: Measures how many operations (insert, delete, substitute) are required to convert one string into another.
- **Dynamic Programming** is used to compute the edit distance efficiently.

Example:

Edit distance between 'kitten' and 'sitting' = 3

(substitute 'k' → 's', substitute 'e' → 'i', insert 'g')

Conclusion

Text processing forms the foundation of NLP tasks. From tokenization to encoding, building vocabulary, and classification, each step is crucial for accurate model performance. Language-independent tokenization ensures scalability across languages, while edit distance and string matching play a key role in error correction. Together, these methods enable robust sentiment classification and other NLP applications.